

Use cases

MapReduce is a good fit for problems that can easily be divided into a number of smaller pieces, which can thus be solved independently. The data is ideally (but not necessarily) in the form of lists, or just a huge chunk of raw information waiting to be processed—be it log files, geospatial data, genetic data to be used in biochemistry, or (of course!) Web pages to be indexed in search engines! Whatever it might be, it is a necessary condition that the final output must not depend on the order in which data is processed.

The use of MapReduce is on the rise in Web analytics, data mining, and various other housekeeping functions in combination with other forms of databases. It is also being used in complex fields ranging from graphics processing in Nvidia's GPUs, to animation and Machine Learning algorithms. I hope all of you have seen the movie Shrek. Guess what? A distributed MapReduce framework was used to render the output frames of the animated movie!

Also, Amazon Elastic MapReduce has allowed this paradigm to gain much more exposure than ever before, and made it a one-click process. Amazon hosts the Hadoop framework running on Amazon EC2 and Amazon S3. It has allowed MapReduce computations without the headache of setting up servers and clusters, making them available as a resource that can be easily rented out and charged for, on an hourly basis.

How to think in terms of 'MapReduce'

MapReduce borrows heavily from the languages of the functional programming model, like Lisp, etc, which are focused on processing lists. So, I suggest you get your concepts in functional programming clear before trying your hand at MapReduce.

Although MapReduce gives programmers with no experience in distributed systems an easy interface, the programmer does have to keep in mind the bandwidth considerations in a cluster, and the amount of data that is being passed around. Carefully implemented MapReduce algorithms can go a long way in improving the performance of a particular cluster. Also, all the computations performed in a MapReduce operation are a batch process, as opposed to SQL, which has an interactive query-like interface. While solving a problem using MapReduce, it is obvious that the problem has to be divided into two functions, i.e., map and reduce:

- The *map* function inputs a series of data streams and processes all the values that follow in the sequence. It takes the initial set of key-value pairs, and in turn, produces an intermediate pair to be passed on to the *reducer*.
- The *reduce* function typically combines all the elements of processed data generated by the *mappers*. Its job is mainly to take a set of intermediate key-

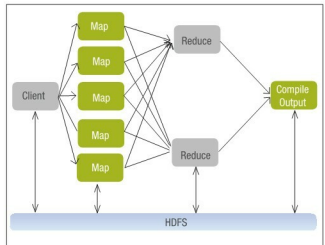


Figure 1: A representational working of a MapReduce program

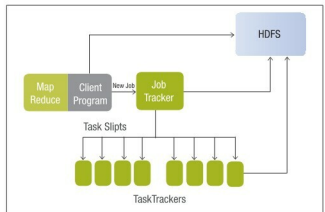


Figure 2: A simplified view of Hadoop's MapReduce job flow

value pairs and output a key-value pair that is basically an aggregate of all the values received by it from the *mapper*.

Combiner functions are sometimes used to combine data on the mapper node, before it goes to the *reducer*. Mostly, the code used to apply a combiner and a reducer function is the same. This allows us to save a lot of data-transfer bandwidth, and can improve efficiency noticeably. But this does not mean that we should be implementing combiners in every case, since if there is not much data to combine, it can take up unnecessary processing power that could be used in a better manner.

There are no limitations on the variety of content that can get passed into a map or a reduce function. It can be anything ranging from simple strings to complex data, to which you can, in turn, again apply MapReduce. Apparently, MapReduce is based upon the popular map/reduce construct from functional programming, but goes into describing the methodology in a much more parallelised fashion.

Also, a lot of work goes into matters such as the various methods of distributing the data among